

Wireshark Packet Analysis — Lab Report

Google Cybersecurity Certificate
Daniel Dela Dzikunu | August 13, 2026

Project Description

A security analyst needs to analyse network traffic to learn what is being sent to and from the systems on the networks they work with. Because packet data is complex, protocol analysers such as Wireshark are designed to find patterns and filter the data down to the traffic that is most relevant to an investigation.

The scenario for this lab is a security analyst investigating traffic to a website. The packet capture file, `sample.pcap`, holds traffic data from a user connecting to an internet site, and the task was to filter that data in order to:

- identify the source and destination IP addresses involved in the web browsing session,
- examine the protocols used when the user connects to the website, and
- analyse individual packets to identify the type of information sent and received by the connected systems.

The work proceeded in five stages: open the capture file and explore the Wireshark interface, open a detailed view of a single packet to examine its protocol and data layers, apply filters to select packets by specific criteria, filter and inspect UDP DNS traffic, and finally filter TCP packet data to search for specific payload text.

Tasks

Task 1 — Explore Data with Wireshark

Opening `sample.pcap` starts Wireshark and lists a large volume of packet traffic. Each packet is a row with seven property columns: **No.** (the packet's index in the file), **Time** (its timestamp), **Source** and **Destination** (the IP addresses), **Protocol**, **Length** (the total length of the packet), and **Info** (Wireshark's interpretation of the payload).

The packets are not all the same colour. Colouring rules give high-level visual cues that classify traffic types quickly, which matters because capture files can hold large amounts of data — light blue for DNS packets, green for a mixture of TCP and HTTP.

Scrolling the list to the first packet whose Info column begins with *Echo (ping) request* gives the answer to the first question.

Result: The protocol of that packet is **ICMP**.

Task 2 — Apply a Basic Filter and Inspect a Packet

The following filter selects traffic associated with one IP address, entered into the *Apply a display filter* box above the packet list:

```
ip.addr == 142.250.1.139
```

The list of displayed packets is now significantly reduced, containing only packets where either the source or the destination matches that address — 16 of the 200, or 8.0% of the capture. Only two colours remain in use: light pink for ICMP packets and light green for TCP, of which HTTP is a subset.

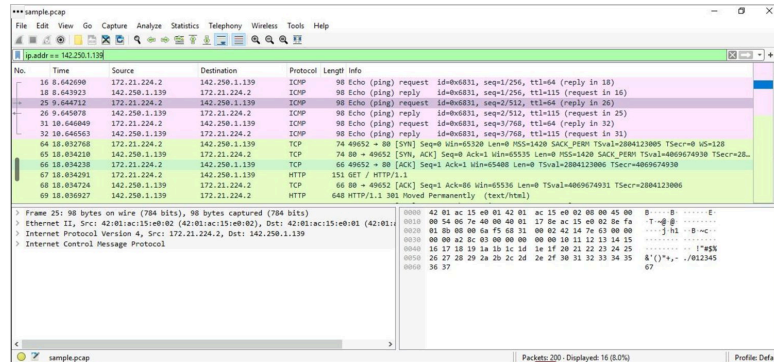


Figure 1 — `ip.addr == 142.250.1.139` reduces the display to 16 packets (8.0%), leaving only light pink ICMP and light green TCP/HTTP traffic.

Double-clicking the first packet listing TCP as its protocol opens the packet details pane. Its upper section holds subtrees carrying Wireshark's analysis of each part of the packet; the lower section shows the raw data in hexadecimal and ASCII, with dots standing in where character data does not apply. Expanding each subtree in turn gives a different layer:

- **Frame** — details about the overall packet, including its length and arrival time.
- **Ethernet II** — the source and destination MAC addresses, and the type of internal protocol the Ethernet packet carries.
- **Internet Protocol Version 4** — the source and destination IP addresses, which match those shown in the main window's summary row, and the internal protocol carried inside the IP packet.
- **Transmission Control Protocol** — the source and destination TCP ports, sequence numbers and flags. The ports match those in the summary row's Info column. Scrolling down and expanding **Flags** gives a detailed view of the flags set in this packet.

Result: The TCP destination port of this packet is **80**.

Task 3 — Use Filters to Select Packets

This task selected packets by where they came from or where they were sent, using either the physical Ethernet MAC address or the IP address. Each filter was applied and cleared in turn:

```
ip.src == 142.250.1.139
ip.dst == 142.250.1.139
eth.addr == 42:01:ac:15:e0:02
```

The source filter returned a shorter list containing only packets that came from 142.250.1.139 — 7 packets, 3.5% of the capture. The destination filter returned only packets sent to that address, 9 packets or 4.5%. The MAC filter is different in kind: it selects traffic to or from one Ethernet MAC address regardless of the other protocols involved.

Opening the first packet in the MAC-filtered list and expanding the Ethernet II subtree shows the filtered address as either the source or the destination. Collapsing it and expanding the Internet Protocol Version 4 subtree, then scrolling to the Time to Live and Protocol fields, shows which internal protocol the packet carries.

Result: The protocol in the Internet Protocol Version 4 subtree of the first packet related to MAC address 42:01:ac:15:e0:02 is **TCP**.

Task 4 — Use Filters to Explore DNS Packets

DNS traffic uses UDP port 53, so the following filter lists DNS queries and responses only:

```
udp.port == 53
```

This returned 22 packets, 11.0% of the capture. Opening the first packet, expanding the Domain Name System subtree and then *Queries*, shows the name being looked up: opensource.google.com. Opening the fourth packet and expanding *Answers* instead shows the queried name together with the addresses associated with it.

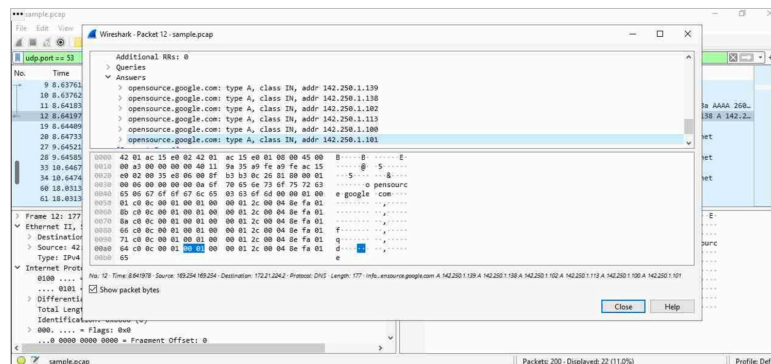


Figure 2 — The expanded Answers section of the DNS response: opensource.google.com with the addresses returned for it.

Result: The address displayed in the expanded Answers section for the opensource.google.com query is **142.250.1.139**.

Task 5 — Use Filters to Explore TCP Packets

TCP port 80 is the default port associated with web traffic, so this filter selects the packets created when the user accessed <http://opensource.google.com>:

```
tcp.port == 80
```

This returned 26 packets, 13.0% of the capture. Opening the first packet in the list gives a destination IP address of 169.254.169.254, and its Frame and Internet Protocol Version 4 subtrees answer the remaining questions.

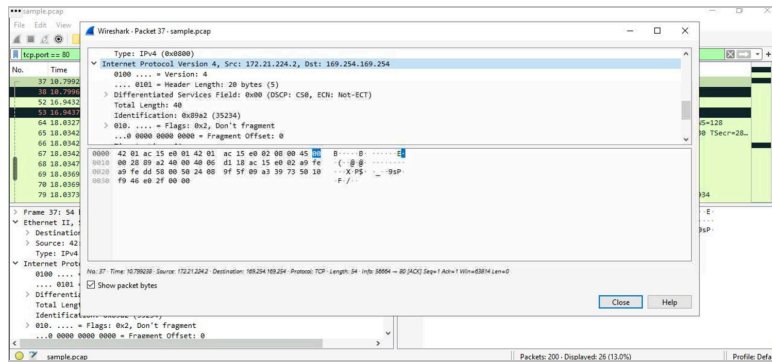


Figure 3 — `tcp.port == 80` returns 26 packets (13.0%). The first packet's IPv4 subtree gives a header length of 20 bytes and a destination of 169.254.169.254.

Result: Time to Live **64**; Frame Length **54** bytes; Header Length **20** bytes; Destination Address **169.254.169.254**.

A final filter searched payload text rather than headers, selecting TCP packets containing a specific string — here, the web requests made with the `curl` command:

```
tcp contains "curl"
```

Summary

Task	Filter	What It Selects	Displayed
1	(none)	The full capture file	200 of 200 (100%)
2	<code>ip.addr == 142.250.1.139</code>	That address as source or destination	16 of 200 (8.0%)
3a	<code>ip.src == 142.250.1.139</code>	Packets that came from that address	7 of 200 (3.5%)
3b	<code>ip.dst == 142.250.1.139</code>	Packets sent to that address	9 of 200 (4.5%)
3c	<code>eth.addr == 42:01:ac:15:e0:02</code>	One MAC address, any protocol	All host traffic
4	<code>udp.port == 53</code>	DNS queries and responses	22 of 200 (11.0%)
5a	<code>tcp.port == 80</code>	Web traffic on the default port	26 of 200 (13.0%)
5b	<code>tcp contains "curl"</code>	Payloads containing that text	curl-issued requests

Conclusion

This lab gave practical experience opening saved packet capture files, viewing high-level packet data, and using filters to inspect detailed packet data — a step toward using network packet analysis tools to examine network traffic.